

BlackBox: A Graph-Based Cyber Governance Architecture for Dynamic Trust Management, Adaptive Containment, and Deception-Driven Recovery

Moses Andrew Raymond
Dept. of Computer Science
and Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India
mosesandrewraymond@gmail.com

Monish M
Dept. of Computer Science
and Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India
monish0924@gmail.com

Mohan Kumar K
Dept. of Computer Science
and Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India
mohan906kee@gmail.com

Dr. Therasa M
Dept. of Computer Science
and Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India

Ms. Shamini M
Dept. of Computer Science
and Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India

Dr. L. Jabasheela
Dept. of Computer Science
and Engineering
Panimalar Engineering College
Chennai, Tamil Nadu, India

Abstract—Most organizations still defend their networks with a patchwork of narrow tools – SIEM platforms for log correlation, EDR for the endpoint, DLP for outbound data, SOAR for playbook automation – each covering one slice of the incident lifecycle and rarely talking to the others. That patchwork buys detection coverage but not command: once a breach begins, nobody owns a live, organization-wide answer to a much simpler question – who and what can still be trusted right now. This paper introduces BlackBox, a Cyber Governance Layer inserted between every user, device, and application on one side and every protected asset on the other, so that access requests are continuously judged rather than merely logged. Its operation follows a repeating five-step cycle – Observe, Govern, Preserve, Contain, Recover – carried out by a Governance Engine, a Dynamic Trust Graph that scores User-Device-Application-Asset relationships as living, mutable edges, a Flight Recorder for forensic logging, a rule-based and explainable Risk Intelligence Engine, a State Preservation Engine, a Cyber Circuit Breaker for narrow containment, an Identity Reset Protocol, and a Deception Engine. Where conventional incident response often isolates a whole server or segment, BlackBox instead severs only the specific trust relationships a compromise has implicated, keeping the rest of the business running while denying an attacker any lasting foothold. We lay out the architecture, give an explicit formal model for how trust and threat are computed, situate the design against Zero Trust Architecture, attack-graph and knowledge-graph research, and cyber-deception literature, and describe the concrete, honestly-scoped state of the working prototype.

Index Terms—cyber governance, zero trust architecture, dynamic trust graph, deception technology, incident response, digital forensics, circuit breaker pattern, security orchestration

I. INTRODUCTION

Most enterprise security stacks are built from specialists rather than generalists. A SIEM platform’s job is to pull logs

together and flag what looks wrong [1]; an EDR agent watches a single machine for behavior that should not be there [2]; a DLP system inspects what data is moving and where it is headed [3]; a SOAR platform waits for one of the above to raise an alert and then runs a scripted response [4]. Each does its narrow job well. The problem is what happens between them: because these tools are bought, deployed, and reasoned about as separate products, an organization’s actual security posture during a live incident is stitched together after the fact from several inconsistent data models, and no single system is ever responsible for maintaining a current, organization-wide answer to who and what should be trusted right now.

That gap shows up worst in the minutes right after a perimeter control fails. Detection-and-alert tooling tells an organization that something happened; it does not actively manage the trust relationships an attacker is now exploiting inside the network. A company can have first-rate logging and still lose practical control of its own environment mid-breach, simply because nothing is continuously deciding, in real time, whether a given user, device, application, or connection should still be allowed to reach a given asset.

BlackBox is an attempt to close that gap directly – not by adding another specialist tool, but by inserting a Cyber Governance Layer between every entity capable of issuing a request (user, device, application) and every asset worth protecting (databases, cloud services, enterprise applications, sensitive documents). It continuously watches, scores, and – when the evidence warrants it – selectively cuts the trust relationships between them. The guiding idea is easy to state and hard to build: even a successful intrusion should never

translate into control. Zero Trust Architecture already applies never-trust-always-verify thinking at the perimeter and per session [5]; BlackBox carries that same discipline forward into a persistent, graph-structured model of trust that keeps evolving throughout an incident, and pairs it with forensic preservation, narrowly-scoped containment, and deception – functions that today live in separate products, if they exist at all.

This paper’s contributions are:

- A single governance-layer architecture in which detection, containment, forensic preservation, deception, and recovery are phases of one running cycle instead of independent tools;
- A Dynamic Trust Graph, with an explicit update rule, that treats User-Device-Application-Asset relationships as scored and mutable rather than fixed by role;
- A Cyber Circuit Breaker that borrows the resilience-engineering circuit-breaker pattern [17] and applies it to security containment, cutting individual trust edges instead of whole networks or servers;
- A five-phase Lockdown Protocol – state preservation, trust severance, asset protection, identity reset, deception deployment – expressed as a single decision algorithm; and
- A grounded comparison against Zero Trust Architecture, attack-graph and knowledge-graph work, deception and moving-target-defense research, and NIST incident-handling guidance, together with an honest account of what is and is not built yet.

Section II surveys the landscape this work sits in – point-solution security, zero trust and trust modeling, graph-based security, deception, and forensic and identity standards. Section III lays out BlackBox’s architecture and operating cycle. Sections IV through IX walk through the Governance Engine and Trust Graph, the observability and risk-scoring layer, preservation and containment, the Lockdown Protocol, the Deception Engine, and the reconstruction and command-center interface. Section X discusses where BlackBox stands and where it falls short. Section XI looks ahead, and Section XII closes out.

II. RELATED WORK

A. Point-Solution Security: SIEM, EDR, DLP, and SOAR

Most security operations centers still run on a SIEM at their core, pulling together and correlating events from across the managed estate [1]; the tooling keeps absorbing big-data analytics techniques, but functionally it remains a detection system built around logs, not a governance system built around decisions [1]. EDR pushes that visibility down to the endpoint, using behavioral analysis to catch what slips past the perimeter [2]. DLP works a different angle, examining the content and context of data wherever it sits – in use, in motion, at rest – to stop it from leaving [3]. SOAR closes the loop by automating the response itself, running a playbook once something else has already raised the alarm; more recent work has paired that

orchestration with honeypots deployed on demand, showing automation can do more than clean up – it can actively deceive [4]. None of these tools are replaced by BlackBox; they are treated as telemetry sources feeding its Flight Recorder, sitting underneath a governance layer they do not themselves provide.

B. Zero Trust and Dynamic Trust Models

NIST’s Zero Trust Architecture publication lays out a policy engine, a policy administrator, and a policy enforcement point that jointly evaluate every request without giving network location any automatic credibility [5]. Adoption surveys, though, point out that in practice most ZTA deployments still treat trust as something decided once, per session, at the moment of access – not as a state that keeps evolving across the whole organization [6]. Trust research in computer science more broadly splits into two camps: credential-based approaches that grant trust from policy, and reputation-based approaches that infer it from a track record of interactions [7]. BlackBox borrows from both: its Governance Engine makes policy-based calls at request time, while its Dynamic Trust Engine keeps a reputation-style score running between requests, rising and falling with observed behavior.

C. Attack Graphs and Knowledge Graphs in Cybersecurity

Treating security as a network of relationships rather than a stream of disconnected events is not a new idea. Attack-graph work captures the paths an adversary could chain together across known vulnerabilities to reach a goal, which supports systematic reasoning about network-wide exposure [8]. More recent efforts build cybersecurity knowledge graphs, linking entities, vulnerabilities, and events into something queryable for situational awareness [9]. BlackBox’s Trust Graph sits near both traditions but serves a different purpose and runs on a different clock: instead of a static, offline picture of what an attacker could theoretically achieve, it is a live structure recording what is presently trusted, updated on every observed event and consulted directly by the Governance Engine for each access decision.

D. Deception-Based Defense and Moving Target Defense

Cyber deception – honeypots, decoy credentials, fabricated infrastructure – works by misleading an attacker and, in the process, learning what they are after; a frequently cited taxonomy sorts deception techniques by which layer they sit at and what goal they serve, from simply buying time to full attribution [10]. Moving Target Defense takes the opposite tack, continually reshaping the attack surface itself – address-space randomization, configuration rotation, and the like – so an attacker never gets the stable picture most attacks rely on [11]; later surveys have mapped that research space and flagged how hard MTD effectiveness is to actually measure [12]. BlackBox’s Deception Engine sits closest to systems that pair orchestration with honeypots deployed on the fly [4], but with one difference: deception here is not a standalone module – it fires directly off Trust Graph and threat-score state.

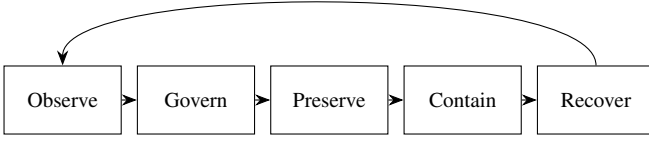


Fig. 1. The continuous Observe–Govern–Preserve–Contain–Recover cycle underlying BlackBox operation. Every stage runs continuously rather than waiting for a triggering alert.

E. Forensic Preservation, Incident Handling, and Identity and Audit Standards

NIST frames incident response as a cycle – preparation, detection and analysis, containment and eradication, post-incident review – and is explicit that evidence has to be captured early enough to support both the technical fix and whatever legal process follows [13]. Separately, digital identity guidance spells out the lifecycle an authenticator or credential goes through, revocation and reissuance included [14]. Tamper-evident logging increasingly leans on blockchain-anchored audit trails, which give cryptographic assurance that a log has not been altered even if part of the logging pipeline itself cannot be fully trusted [15]. BlackBox’s State Preservation Engine, Identity Reset Protocol, and Incident Reconstruction Engine map onto these three bodies of guidance respectively, and are covered in Sections VI, VII, and IX.

III. BLACKBOX SYSTEM ARCHITECTURE

A. Design Philosophy

BlackBox does not run as a one-shot detection pipeline; it runs as a loop. Fig. 1 lays out the five stages that repeat for as long as the system is live: Observe (telemetry keeps flowing in from identity, device, application, and asset layers, continuously), Govern (every request is checked against the current trust and policy state before it is let through), Preserve (a forensic snapshot is taken before any containment step, so the evidence predates anything an attacker might try to erase), Contain (the Cyber Circuit Breaker cuts only the specific trust edges a compromise has implicated), and Recover (identity reset, deception deployment, and automatic incident reconstruction bring the system back to safe operation, rebuilding trust gradually rather than resetting it to zero). Systems that wait for a signature or an anomaly to fire an alert are, by construction, idle most of the time; every stage of this cycle, by contrast, runs continuously, across the whole environment at once.

B. High-Level Request Flow

Fig. 2 shows where BlackBox physically sits. Every request from a user, device, or application passes through the BlackBox Cyber Governance Layer before it can reach a critical asset – a database, a cloud service, an enterprise application, a sensitive document. That placement is what makes governance enforceable rather than aspirational: since nothing has a route to a protected asset that skips BlackBox, the Governance Engine’s trust and policy checks apply uniformly, rather than

depending on every downstream system to independently get the enforcement right.

IV. GOVERNANCE ENGINE AND DYNAMIC TRUST GRAPH

A. Governance Engine

The Governance Engine makes the actual access decisions. For each request it looks at who is asking, what device and application are involved, what asset is being targeted, whether the behavior fits the usual pattern for that combination, and the current trust and threat scores tied to those entities. From that, it returns one of four verdicts: Allow, Restrict, Warn, or Block. The decision space is kept deliberately small and legible – every verdict has to be traceable back to the trust and threat inputs that produced it, rather than buried inside an opaque score.

B. Dynamic Trust Engine

Static roles do not drive permissions here. Instead, BlackBox keeps a running trust score for every user, device, application, and relationship in the system. Let $T_i(t) \in [0, 100]$ be the trust score of entity or relationship i at step t . An earlier version of this model gave every legitimate interaction and every violation category the same fixed weight regardless of circumstance, and simply left T_i untouched whenever i went quiet. We replaced that with an update rule that saturates, accounts for what is actually at risk, and drifts back toward normal on its own:

$$T_i(t+1) = \text{clip} \left(T_i(t) + \eta_r R_i(t) \left(1 - \frac{T_i(t)}{100} \right) - \sum_{k \in E_i(t)} \eta_p^{(k)} c_a(k) - \lambda (T_i(t) - T_0) \mathbb{1}_{\text{idle}}, 0, 100 \right) \quad (1)$$

Here $R_i(t) \in \{0, 1\}$ flags whether i ’s interaction at t looked legitimate; $E_i(t)$ is the set of suspicious-event categories seen for i at t – an unfamiliar login location, an unrecognized device, an oversized file transfer, a policy breach, contact with a deception asset; $\eta_p^{(k)}$ is category k ’s base penalty; $c_a(k) \in \{1, 1.5, 2, 3\}$ scales that penalty by how sensitive the involved asset is (Public through Critical – see Section IV-E), so the same infraction costs more when it touches something more valuable; η_r is a small reward rate; $\lambda (T_i(t) - T_0) \mathbb{1}_{\text{idle}}$ nudges an inactive entity’s score back toward a neutral baseline T_0 (e.g., 70) instead of letting it sit frozen indefinitely; and $\text{clip}(x, a, b)$ keeps x inside $[a, b]$. Two properties fall out of this naturally: the $(1 - T_i(t)/100)$ term means a reward matters less the closer T_i already is to 100, so trust eases toward its ceiling instead of hitting a hard wall; and the drift term makes trust a genuinely time-aware quantity rather than something that only moves when an event fires. None of the constants here – η_r , $\eta_p^{(k)}$, λ , T_0 – are empirically tuned yet; that calibration work is part of what Section X-B describes as still ahead.

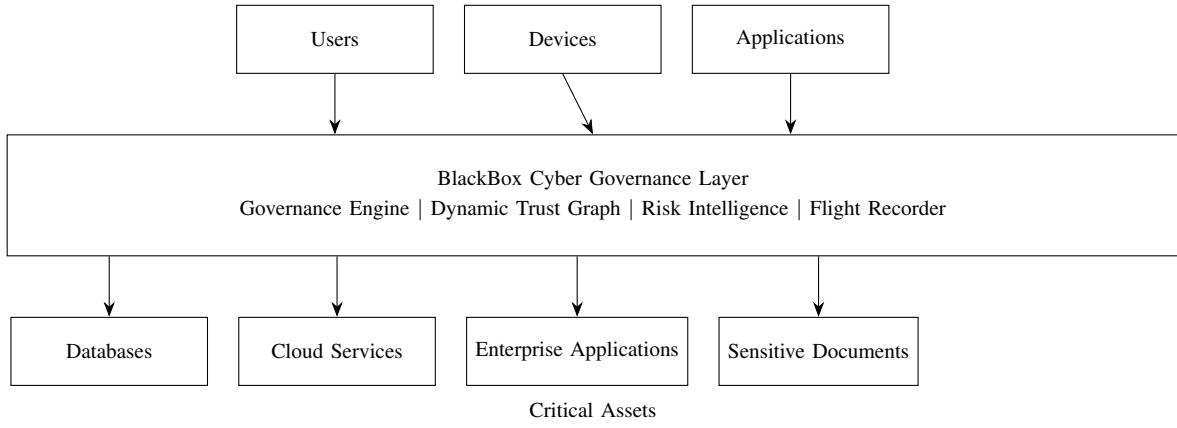


Fig. 2. High-level request flow through the BlackBox Cyber Governance Layer. Every access request from users, devices, and applications is evaluated by the governance layer before it can reach a critical organizational asset.

C. Trust Graph Model

The Trust Graph is what everything else in BlackBox is built on top of. Rather than treating each security event as its own isolated fact, it represents the chain $\text{User} \rightarrow \text{Device} \rightarrow \text{Application} \rightarrow \text{Asset}$ as a graph whose edges each carry a trust score (from Eq. (1)), a status – Active, Monitored, Compromised, or Severed – a record of past activity, and a risk level. That places it near two existing research threads: attack-graph modeling, which maps out how an attacker could chain vulnerabilities to reach a target [8], and cybersecurity knowledge graphs, which connect entities and events for situational awareness [9]. What sets the Trust Graph apart from both is that it is live: rather than a static analysis of theoretical exposure, it is a running record of what is trusted right now. Administrators see it directly through the Governance Command Center (Section IX-B), and – this matters for the containment mechanism in Section VI-B – BlackBox can act on one edge in isolation, without having to reason about the graph as a whole.

D. Enterprise Access Manager

The Enterprise Access Manager (EAM) is a separate concern from the Trust Graph: it is conventional role-based access control, deciding which identities can do what to which resources under the policy currently in force. The Trust Graph’s question is who trusts whom right now; the EAM’s question is what is this entity allowed to do. A Finance Manager role, for instance, might be cleared for payroll records but blocked from infrastructure credentials; touching the latter gets flagged as a violation. The two systems communicate in one direction only – an EAM violation becomes a suspicious-event category $k \in E_i(t)$ that feeds into the Dynamic Trust Engine (Section IV-B, Eq. (1)) and pulls down the offending identity’s trust score, but authorization itself is never derived from the Trust Graph. Keeping RBAC/policy and trust/reputation as separate layers, rather than folding them into a single number, matches how enterprise security architectures generally treat these two concerns.

E. Asset Classification Engine

Every asset in the environment is sorted automatically into one of four tiers: Public, Internal, Confidential, Critical. An employee handbook would usually land as Internal; a payroll database or a customer-records store would be Critical; a source-code repository would be Confidential. That classification is not just a label – it feeds governance policy, risk calculation, the order in which the Cyber Circuit Breaker acts (Section VI-B), and lockdown decisions (Section VII). Concretely, it sets the criticality multiplier $c_a(k)$ in Eq. (1) (Public = 1, Internal = 1.5, Confidential = 2, Critical = 3), so touching a Critical asset costs an entity more trust – and pushes the threat score up faster – than an equivalent event on something Public.

V. OBSERVABILITY AND RISK INTELLIGENCE

A. Flight Recorder Engine

The Flight Recorder borrows its name from aviation’s flight-data recorders, and does much the same job for security: it keeps a continuous log of logins, file access, creation, modification, and deletion, process execution, USB activity, network activity, admin actions, and policy violations. Each record carries a timestamp, user, device, application, asset, threat state, and risk delta. That continuous record is what makes the Incident Reconstruction Engine in Section IX-A possible at all, and it turns NIST’s evidence-collection guidance [13] into something the system does by default, rather than something assembled after the fact.

B. Risk Intelligence Engine

The Risk Intelligence Engine keeps a running threat score for the organization. An earlier version of this score simply summed rule indicators with no sense of time – an indicator from ten minutes back counted exactly the same as one from right now, and indicators firing together got no more weight than the same set spread across unrelated days. We changed that so the score decays with age and rewards indicators that show up together:

$$S(t) = \gamma S(t-1) + \sum_{k=1}^K w_k I_k(t) + \beta \binom{|A(t)|}{2} \quad (2)$$

$I_k(t) \in \{0, 1\}$ is a binary flag for rule k – excessive file access, an oversized transfer, an unfamiliar device, a login at an odd hour, a policy breach, a touch on a honeypot – and w_k is how severely an analyst has weighted that rule. $\gamma \in (0, 1)$ decays older activity out of $S(t)$ instead of letting it persist indefinitely, and $A(t) = \{k : I_k(t) = 1\}$ is the set of distinct rule categories firing at t , so the term $\beta \binom{|A(t)|}{2}$ rewards several categories showing up together – an unfamiliar device plus an odd hour plus a large transfer, all at once, is a stronger signal than the same three spread across separate days, which lines up with how attack-graph research treats chained conditions as more significant than isolated ones [8]. Because $S(t)$ depends only on $S(t-1)$, updating it costs constant time per event rather than resumming a sliding window. $S(t)$ is then mapped to one of four bands – Green, Yellow, Orange, Red – by threshold. Every I_k maps to a named, auditable rule rather than a feature buried inside an opaque model, so an analyst can always trace a shift in $S(t)$ back to exactly what caused it – the kind of explainability recent intrusion-detection work argues is necessary for analyst trust and for the system to hold up under regulatory scrutiny [16].

The earlier model tracked a per-entity trust score $T_i(t)$ (Section IV-B) and a global threat score $S(t)$ with no link between them – two entities behind an identical event would score identically regardless of how much trust either had built up. We tie the two together with an effective score used for the actual governance and lockdown decision (Section VII), in place of raw $S(t)$:

$$S_{\text{eff},i}(t) = S(t) \left(1 + \kappa \left(1 - \frac{T_i(t)}{100} \right) \right) \quad (3)$$

$\kappa \geq 0$ sets how strongly low trust amplifies the effective threat level. A well-established, high-trust entity needs more evidence before the same organizational threat score restricts it; a low-trust entity is flagged sooner. That is the risk-adaptive access idea Zero Trust guidance treats as central [5], turned here into a single multiplicative correction rather than a qualitative policy statement.

VI. PRESERVATION AND CONTAINMENT

A. State Preservation Engine

Before any containment step runs, BlackBox captures a full forensic snapshot: running processes and the process tree, memory usage, active users, network connections, open ports, recent file activity, policy violations, the Trust Graph’s current state, the threat score, and system metadata. The principle behind it is simple – evidence has to be captured before an attacker gets the chance to destroy it, not pieced back together afterward from whatever logs happened to survive, which is exactly the forensic-readiness point incident-handling guidance keeps making [13].

B. Cyber Circuit Breaker Architecture

The usual move in incident response is to disconnect an entire server or network segment once something looks compromised – effective, but blunt: everyone and everything sharing that server or segment gets cut off along with the actual attacker. BlackBox takes the circuit-breaker pattern from resilience engineering instead – originally built to stop cascading failures between software services by tripping only the one dependency that is actually failing [17] – and applies it to containment. A Cyber Circuit Breaker trips only the specific trust edges a compromise touches: one User→Asset relationship, one Device→Asset relationship, one Application→Asset relationship, a remote-access session, an external transfer, or a single API connection. Because the Trust Graph represents these as individually addressable edges (Section IV-C), the breaker can isolate exactly the compromised path and leave everything else – every other user, device, and process – untouched.

VII. BLACKBOX LOCKDOWN PROTOCOL

Once the threat score $S(t)$ crosses into the Red band, BlackBox runs the Lockdown Protocol (BLP) automatically, in five phases: State Preservation (the snapshot from Section VI-A), Trust Severance (tripping Cyber Circuit Breakers on the implicated edges), Asset Protection (tightening access and policy on the affected assets), Identity Reset (below), and Deception Deployment (bringing up the Deception Engine from Section VIII). Algorithm 1 spells out exactly when each phase fires, building on the Governance Engine’s per-request decision (Section IV-A) with the graph-wide lockdown logic layered on top.

Algorithm 1 Lockdown Trigger Evaluation

Require: Event e ; entity i ; Trust Graph G ; threat score S ; thresholds $\tau_y < \tau_o < \tau_r$

Ensure: Governance decision d ; lockdown flag ℓ

- 1: $S \leftarrow \gamma S + \Delta(e)$ {Eq. (2): decay, add indicators + correlation bonus}
 - 2: Update T_i for every entity i touched by e {Eq. (1)}
 - 3: $S_{\text{eff},i} \leftarrow S(1 + \kappa(1 - T_i/100))$ {Eq. (3)}
 - 4: **if** $S_{\text{eff},i} \geq \tau_r$ **then**
 - 5: $\ell \leftarrow \text{true}$
 - 6: StatePreservation
 - 7: CircuitBreaker(compromised edges of G)
 - 8: IdentityReset(affected entities)
 - 9: DeceptionDeployment
 - 10: $d \leftarrow \text{Block}$
 - 11: **else if** $S_{\text{eff},i} \geq \tau_o$ **then**
 - 12: $d \leftarrow \text{Restrict}$
 - 13: **else if** $S_{\text{eff},i} \geq \tau_y$ **then**
 - 14: $d \leftarrow \text{Warn}$
 - 15: **else**
 - 16: $d \leftarrow \text{Allow}$
 - 17: **end if**
 - 18: **return** (d, ℓ)
-

A. Identity Reset Protocol

The fourth phase simulates password rotation, session revocation, token invalidation, API key rotation, and credential-reset for every identity the incident has implicated. These map directly onto the authenticator lifecycle – issue, use, revoke – that NIST’s digital identity guidelines already define [14]; effectively, the Identity Reset Protocol forces an accelerated pass through that lifecycle for exactly the identities the Trust Graph has flagged, instead of resetting credentials organization-wide.

VIII. DECEPTION ENGINE

Cutting off a suspected attacker immediately tells them they have been caught and ends any chance to learn from what they were doing. BlackBox’s alternative is to redirect that activity toward controlled deception assets instead – honeypots, decoy databases, fake credentials, synthetic confidential documents. The point is fourfold: slow the attacker down, learn what they are after, gather forensic intelligence, and keep the real data untouched. That follows the broader deception literature’s approach of classifying techniques by layer and objective [10], and complements Moving Target Defense’s strategy of denying attackers a stable picture rather than handing them a false one [11], [12]. Architecturally, BlackBox’s implementation sits closest to systems that pair SOAR-style orchestration with honeypots deployed dynamically [4]; what is different here is that deception is not its own subsystem – it is simply the fifth phase of the same Lockdown Protocol from Section VII, fired directly by Trust Graph and threat-score state.

IX. INCIDENT RECONSTRUCTION AND GOVERNANCE COMMAND CENTER

A. Incident Reconstruction Engine

Once containment finishes, BlackBox assembles a full forensic report on its own: a timeline, the threat-escalation history, trust-score changes, circuit-breaker activations, state snapshots, deception events, identity-reset actions, and a summary. That turns the post-incident “lessons learned” step NIST treats as part of the response cycle [13] into something automatic rather than optional paperwork, and the report itself can be anchored to a tamper-evident log using blockchain-based audit-trail techniques for later regulatory or legal review [15].

B. Governance Command Center

BlackBox’s interface is built as a governance command center, not a log viewer. It shows the current threat state and score, trust score and overall trust health, a live Trust Graph visualization, a plain-language governance narrative, an event feed, circuit-breaker status, identity-reset status, a state-preservation viewer, an asset registry, and an incident timeline. A conventional SIEM dashboard puts raw logs and correlation rules front and center [1]; the Governance Command Center instead puts trust relationships and the decisions made about them front and center, on the premise that an analyst deciding how to respond needs to see what is currently trusted and why – not just what got logged.

TABLE I
QUALITATIVE COMPARISON OF BLACKBOX WITH CONVENTIONAL SECURITY PARADIGMS

Dimension	SIEM / SOAR [1], [4]	ZTA [5], [6]	BlackBox
Primary scope	Log correlation, playbook response	Identity-centric access control	Unified governance across identity, device, app, and asset
Trust model	Static rules	Continuous, per-session	Continuous, graph-based, per-relationship
Containment unit	Host or network segment	Session request or	Individual trust edge
Forensic snapshot	Log-based, largely after the fact	Not explicitly defined	Captured before containment acts
Deception	Occasionally bolted on	Not addressed	Native fifth phase of lockdown

X. DISCUSSION

A. Comparative Positioning

Table I lays out, at a qualitative level, where BlackBox sits relative to the point-solution and zero-trust paradigms covered in Section II. This is a conceptual comparison, not an empirical one – no head-to-head benchmark exists yet, and the table should be read as a statement of intent rather than a measured result.

B. Prototype Scope and Implementation Plan

BlackBox is being built as a final-year academic project with an eye toward a longer product life afterward. The working implementation is a web application: React, Vite, and TypeScript on the frontend (styled with Tailwind CSS and shadcn/ui), talking to an Express/TypeScript backend over a REST API and WebSockets. That backend hosts the Governance, Trust, Risk, Lockdown, Flight Recorder, and Deception logic; state lives in PostgreSQL via the Drizzle ORM, with a handful of latency-sensitive values – the live risk score, circuit-breaker status – kept in memory instead. Progress has been incremental: the dashboard, Trust Graph, Risk Engine, Trust Score, Attack Simulator, Enterprise Access Manager, Circuit Breakers, and live WebSocket updates all work; the Governance Engine, Flight Recorder, State Preservation, Deception Engine, and Recovery phase are at different stages of completeness; and several response actions this paper describes – password rotation, session revocation, memory/process capture, network isolation – currently exist as visual simulations rather than actions with any real effect on a host or network. Given all that, this paper presents an architecture and a formal model, not benchmarked results.

Beyond η_r , $\eta_p^{(k)}$, λ , and T_0 (Eq. (1)), the decay factor γ and correlation weight β (Eq. (2)) and the trust-sensitivity constant κ (Eq. (3)) all still need calibration against real or replayed attack traffic rather than being set by hand. Once the simulated

pieces are replaced with real effectors, the planned evaluation covers: containment latency from threat-score escalation to circuit-breaker activation; false-positive and false-negative rates for the rule-based indicators in Eq. (2) against labeled attack traffic; how complete the forensic snapshots are against ground truth; and how much less disruption unaffected users see compared to conventional whole-network isolation.

C. Limitations

Four things are worth stating plainly. First, the explainable, rule-based risk scoring in Section V-B trades some detection sophistication for auditability – a tension the explainable-intrusion-detection literature keeps returning to [16]. Second, the Trust Graph and Governance Engine are themselves high-value targets: whoever compromises the governance layer compromises trust computation itself, so BlackBox’s own components have to be classified and protected as Critical assets (Section IV-E), not treated as infrastructure that is automatically trusted. Third, deception assets need continuous upkeep to stay convincing against attackers sophisticated enough to probe specifically for honeypot tells [10]. Fourth, and most fundamentally, what is described here is an architecture and a formal model that has not been validated empirically yet; the claims in this paper are about design and feasibility, not measured performance.

XI. FUTURE SCOPE

Planned extensions include machine-learning-based behavioral analytics layered on top of the current rule-based scoring – an area where multimodal deep learning, which has already shown its value fusing heterogeneous data sources for prediction in domains such as clinical diagnosis [18], offers a useful template for combining device, network, and behavioral telemetry into a single risk signal; deeper integration with Zero Trust Architecture deployments [5], [6]; cloud-native deployment; hooks into external threat-intelligence platforms; blockchain-based audit trails for the Incident Reconstruction Engine [15]; digital-twin attack simulation so the Lockdown Protocol can be tested without touching production assets; predictive cyber-risk modeling; cross-enterprise trust federation; steganalysis techniques adapted from machine-learning-based steganography research [19] to help the Deception and Risk Intelligence Engines catch covert exfiltration channels that plain content inspection would miss; natural-language techniques – including machine reading comprehension methods for extracting answers from unstructured text [20] – to let analysts query the Flight Recorder’s logs and the incident timeline directly, in plain language, rather than through a fixed dashboard; and increasingly autonomous recovery orchestration that reduces the manual steps still required after the Deception Deployment phase.

XII. CONCLUSION

This paper laid out BlackBox, a graph-based Cyber Governance Layer built around the idea that security should mean continuous trust management, not a collection of

point solutions bolted together. By representing User-Device-Application-Asset relationships as a live, scored Trust Graph, and by folding a Governance Engine, forensic preservation, a fine-grained Cyber Circuit Breaker, an Identity Reset Protocol, a Deception Engine, and automatic incident reconstruction into one five-phase Lockdown Protocol, BlackBox is meant to let an organization keep operational control across the full arc of an attack – observing, governing, preserving, containing, recovering – without disrupting the people who were not involved. We gave explicit formal models for how trust and threat get computed, placed the design against Zero Trust Architecture, attack-graph and knowledge-graph research, deception and moving-target-defense literature, and NIST’s forensic, incident-handling, and identity standards, and described exactly where the prototype stands today – a React/Express/PostgreSQL implementation with several components still simulated – and what has to happen before its claims can be checked against real data.

REFERENCES

- [1] G. Gonzalez-Granadillo, S. González-Zarzosa, and R. Diaz, “Security information and event management (SIEM): Analysis, trends, and usage in critical infrastructures,” *Sensors*, vol. 21, no. 14, Art. no. 4759, 2021.
- [2] H. Kaur et al., “Evolution of endpoint detection and response (EDR) in cyber security: A comprehensive review,” *E3S Web of Conferences*, vol. 86, 2024.
- [3] S. Alneyadi, E. Sithirasanen, and V. Muthukumarasamy, “A survey on data leakage prevention systems,” *Journal of Network and Computer Applications*, vol. 62, pp. 137–152, Feb. 2016.
- [4] U. Bartwal, S. Mukhopadhyay, R. Negi, and S. Shukla, “Security orchestration, automation, and response engine for deployment of behavioural honeypots,” in *Proc. IEEE Conf. Dependable and Secure Computing (DSC)*, 2022.
- [5] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” NIST Special Publication 800-207, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2020.
- [6] N. F. Syed, S. W. Shah, A. Shaghagh, A. Anwar, Z. Baig, and R. Doss, “Zero trust architecture (ZTA): A comprehensive survey,” *IEEE Access*, vol. 10, pp. 57143–57179, 2022.
- [7] D. Artz and Y. Gil, “A survey of trust in computer science and the Semantic Web,” *Web Semantics: Science, Services and Agents on the World Wide Web*, vol. 5, no. 2, pp. 58–71, 2007.
- [8] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, “Automated generation and analysis of attack graphs,” in *Proc. IEEE Symp. Security and Privacy*, 2002, pp. 273–284.
- [9] Y. Han and A. Li, “A survey on cybersecurity knowledge graph construction,” *Computers & Security*, 2023, Art. no. 103524.
- [10] X. Han, N. Kheir, and D. Balzarotti, “Deception techniques in computer security: A research perspective,” *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–36, Jul. 2018.
- [11] S. Jajodia, A. K. Ghosh, V. Swarup, C. Wang, and X. S. Wang, Eds., *Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats*, *Advances in Information Security*, vol. 54. New York, NY, USA: Springer, 2011.
- [12] J.-H. Cho, D. P. Sharma, H. Alavizadeh, S. Yoon, N. Ben-Asher, T. J. Moore, D. S. Kim, H. Lim, and F. F. Nelson, “Toward proactive, adaptive defense: A survey on moving target defense,” *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, pp. 709–745, 2020.
- [13] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, “Computer security incident handling guide,” NIST Special Publication 800-61 Revision 2, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2012.
- [14] P. A. Grassi, J. L. Fenton, E. M. Newton, R. A. Perlner, A. R. Regenscheid, W. E. Burr, J. P. Richer, N. B. Lefkowitz, J. M. Danker, Y.-Y. Choong, K. K. Greene, and M. F. Theofanos, “Digital identity guidelines: Authentication and lifecycle management,” NIST Special Publication 800-63B, National Institute of Standards and Technology, Gaithersburg, MD, USA, Jun. 2017.

- [15] A. Ahmad, M. Saad, M. Al Ghamdi, D. Nyang, and D. Mohaisen, "BlockTrail: A service for secure and transparent blockchain-driven audit trails," *IEEE Systems Journal*, vol. 16, no. 1, pp. 1367–1378, Mar. 2022.
- [16] S. Neupane, J. Ables, W. Anderson, S. Mittal, S. Rahimi, I. Banicescu, and M. Seale, "Explainable intrusion detection systems (X-IDS): A survey of current methods, challenges, and opportunities," *IEEE Access*, vol. 10, pp. 112392–112415, 2022.
- [17] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.
- [18] Shamini, M., et al. "A Multimodal Deep Learning Approach for Predicting Dementia's Disease Progression Using MRI Clinical Data." 2025 International Conference on Data Science, Agents & Artificial Intelligence (ICDAAI), IEEE, 2025.
- [19] Murugavalli, S., L. Jabasheela, and V. Anitha. "Ceaseless Steganographic Approaches in Machine Learning." *Measurement: Sensors*, vol. 25, 2023, p. 100622.
- [20] Therasa, M., and G. Mathivanan. "Survey of Machine Reading Comprehension Models and Its Evaluation Metrics." 2022 6th International Conference on Computing Methodologies and Communication (ICCMC), IEEE, 2022.